SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE:CSA1741

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192312070**
Name	**Balaguru S**

COURSE OUTCOME COVERED

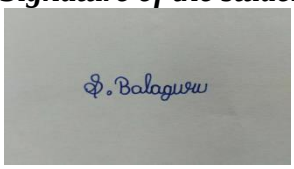
CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weight age: 20%

Code of Conduct:

I Balaguru S (192312070) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5
Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

1. **A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:**

Constraints:

i. **D1 cannot work Night shift**

ii. **D2 must work before D3 (Morning < Afternoon < Night)**

iii. **Only one doctor per shift**

iv. **D3 cannot work Morning**

v. **All doctors must be assigned exactly one shift**

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Doctor Shift Assignment Using Backtracking Search

1. Problem Understanding (6 Marks)

Given:

Doctors: D1, D2, D3

Shifts: Morning, Afternoon, Night

Constraints:

1. D1 cannot work the Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor per shift.
4. D3 cannot work the Morning shift.
5. All doctors must be assigned exactly one shift.

Objective: Apply Backtracking Search to find a valid schedule satisfying all the constraints.

2. Constraint Analysis (6 Marks)

Constraint	**Condition**
C1	D1 ≠ Night
C2	Shift(D2) < Shift(D3)
C3	Each shift is assigned to only one doctor
C4	D3 ≠ Morning
C5	Each doctor receives exactly one shift

The shift order is Morning < Afternoon < Night. D2 must work before D3, and D3 cannot work Morning. Therefore, D2 and D3 can be assigned Morning–Afternoon, Morning–Night, or Afternoon–Night, respectively.

3. Solution Development Using Backtracking (7.5 Marks)

Intermediate Steps and Constraint Checks

Step	Assignment	Constraint Check	Action
1	D1 → Night	Violates C1 because D1 cannot work Night.	Reject
2	D1 → Morning	Satisfies C1.	Accept
3	D2 → Night	No later shift is available for D3, violating C2.	Reject and backtrack
4	D2 → Afternoon	D2 can work before D3.	Accept
5	D3 → Night	Satisfies C2, C3, and C4.	Accept
6	Check all assignments	Every doctor has one unique shift, and all constraints are satisfied.	Success

4. Application of Concepts (6 Marks)

Python Program Using Backtracking Search

```
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]
assignment = {}
def is_valid(doctor, shift):
    if shift in assignment.values():
        return False
    if doctor == "D1" and shift == "Night":
        return False
    if doctor == "D3" and shift == "Morning":
        return False
    if doctor == "D2" and "D3" in assignment:
        if shifts.index(shift) >= shifts.index(assignment["D3"]):
            return False
    if doctor == "D3" and "D2" in assignment:
        if shifts.index(assignment["D2"]) >= shifts.index(shift):
            return False
    return True
def backtrack(index):
```

```
    if index == len(doctors):
        return True
    doctor = doctors[index]
    for shift in shifts:
        if is_valid(doctor, shift):
            assignment[doctor] = shift
            print("Assigned:", doctor, "->", shift)
            if backtrack(index + 1):
                return True
            print("Backtracking:", doctor, "->", shift)
            del assignment[doctor]
    return False
if backtrack(0):
    print("\nFinal Valid Schedule:")
    for shift in shifts:
        for doctor in doctors:
            if assignment[doctor] == shift:
                print(shift, ":", doctor)
else:
    print("No valid schedule found")
```

Expected Output

Assigned: D1 -> Morning

Assigned: D2 -> Afternoon

Assigned: D3 -> Night

Final Valid Schedule:

Morning : D1

Afternoon : D2

Night : D3

5. Justification (4.5 Marks)

The final schedule satisfies all five constraints:

1. D1 is assigned to Morning and does not work Night.

2. D2 is assigned to Afternoon and works before D3.
3. D3 is assigned to Night and does not work Morning.
4. Each shift has exactly one doctor.
5. All doctors are assigned exactly one shift.

Backtracking Search explores possible assignments, rejects invalid choices, and returns to previous decisions when necessary. This ensures that the final schedule satisfies all constraints.

Result: A valid schedule is obtained using Backtracking Search: D1 → Morning, D2 → Afternoon, and D3 → Night.

---

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. **Movement allowed: Up, Down, Left, Right**

ii. **Cost of each move = 1**

iii. **Cannot pass through obstacles**

iv. **Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Robot Pathfinding in a 5×5 Grid Using BFS Search Algorithm

1. Problem Understanding (6 Marks)

Given:

- Grid size: 5 × 5
- Start position: S = (1, 1)
- Goal position: G = (5, 5)
- Obstacles: (2, 2), (2, 4), (3, 2), (4, 4)
- Allowed movements: Up, Down, Left, Right
- Cost of each move = 1
- Heuristic: Manhattan Distance

Objective: Find the optimal path from Start to Goal while avoiding obstacles and minimizing the total movement cost.

2. Constraint Analysis (6 Marks)

The robot must follow these constraints:

1. It can move only Up, Down, Left, or Right.
2. Each movement has a cost of 1.
3. The robot cannot enter obstacle cells.
4. The robot must remain within the 5×5 grid.
5. Manhattan Distance is calculated using:

$h(n) = |x_1 - x_2| + |y_1 - y_2|$

6. The optimal path is the path with the minimum number of moves.

Grid Representation:

S . . . .

. # . # .

. # . . .

. . . # .

. . . . G

S = Start, G = Goal, # = Obstacle, . = Free cell.

3. Solution Development (7.5 Marks)

Breadth-First Search (BFS) explores nodes level by level using a FIFO queue. Since every movement costs 1, BFS guarantees an optimal path.

The Manhattan Distance heuristic is used to estimate the distance to the goal. However, ordinary BFS does not use the heuristic to prioritize nodes; it is calculated for reference. If nodes are prioritized using both path cost and heuristic, the algorithm becomes A* Search.

Initial values:

- Start: (1, 1)
- Goal: (5, 5)
- Initial queue: [(1, 1)]
- Initial path cost: 0
- Initial Manhattan Distance: $|1 - 5| + |1 - 5| = 8$

Step-by-Step Node Expansion and Queue Updates

Coordinates are represented as (row, column). Neighbors are considered in the order Right, Down, Left, Up. Already visited nodes and obstacles are skipped.

Step	Expanded Node	Newly Discovered Nodes	Queue After Expansion
1	(1,1)	(1,2), (2,1)	[(1,2), (2,1)]
2	(1,2)	(1,3)	[(2,1), (1,3)]
3	(2,1)	(3,1)	[(1,3), (3,1)]
4	(1,3)	(1,4), (2,3)	[(3,1), (1,4), (2,3)]
5	(3,1)	(4,1)	[(1,4), (2,3), (4,1)]
6	(1,4)	(1,5)	[(2,3), (4,1), (1,5)]
7	(2,3)	(3,3)	[(4,1), (1,5), (3,3)]
8	(4,1)	(4,2)	[(1,5), (3,3), (4,2)]
9	(1,5)	(2,5)	[(3,3), (4,2), (2,5)]
10	(3,3)	(3,4), (4,3)	[(4,2), (2,5), (3,4), (4,3)]
11	(4,2)	(5,2)	[(2,5), (3,4), (4,3), (5,2)]
12	(2,5)	(3,5)	[(3,4), (4,3), (5,2), (3,5)]
13	(3,4)	None	[(4,3), (5,2), (3,5)]
14	(4,3)	(5,3)	[(5,2), (3,5), (5,3)]
15	(5,2)	None	[(3,5), (5,3)]
16	(3,5)	(4,5)	[(5,3), (4,5)]
17	(5,3)	(5,4)	[(4,5), (5,4)]
18	(4,5)	(5,5)	[(5,4), (5,5)]
19	(5,4)	None	[(5,5)]
20	(5,5)	Goal reached	[]

The goal is reached after 20 node expansions in this traversal order.

Optimal Path

Using the recorded parent nodes, one shortest path is:

(1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5)

Path Visualization:

S * * * *

. # . # *

. # . . *

. . . # *

. . . . G

Here, * represents the chosen path.

4. Application of Concepts (6 Marks)

Python Program Using BFS

```
from collections import deque
grid = [
    ['S', '.', '.', '.', '.'],
    ['.', '#', '.', '#', '.'],
    ['.', '#', '.', '.', '.'],
    ['.', '.', '.', '#', '.'],
    ['.', '.', '.', '.', 'G']
]
start = (0, 0)
goal = (4, 4)
directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
queue = deque([start])
parent = {start: None}
distance = {start: 0}
step = 1
while queue:
    current = queue.popleft()
    print("Step", step, "Expanded:", (current[0] + 1, current[1] + 1))
    step += 1
    if current == goal:
        break
```

```
    for dr, dc in directions:
        nr, nc = current[0] + dr, current[1] + dc
        neighbor = (nr, nc)
        if 0 <= nr < 5 and 0 <= nc < 5:
            if grid[nr][nc] != '#' and neighbor not in parent:
                parent[neighbor] = current
                distance[neighbor] = distance[current] + 1
                queue.append(neighbor)
    print("Queue:", [(r + 1, c + 1) for r, c in queue])
if goal in parent:
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    print("\nOptimal Path:")
    print(" -> ".join(str((r + 1, c + 1)) for r, c in path))
    print("Optimal Cost:", distance[goal])
    print("Manhattan Distance at Start:", abs(start[0] - goal[0]) + abs(start[1] - goal[1]))
else:
    print("No path found")
```

5. Justification (4.5 Marks)

BFS explores every node at a given distance before exploring nodes at a greater distance. Since each move has an equal cost of 1, the first time the goal is reached, the recorded path is optimal.

Manhattan Distance estimates the minimum number of horizontal and vertical moves required to reach the goal, ignoring obstacles. It is 8 at the start. Obstacles may force the robot to take a longer route.

Final Result

- **Algorithm:** Breadth-First Search (BFS)
- **Grid size:** 5 × 5
- **Start:** (1, 1)
- **Goal:** (5, 5)

- **Optimal path:** (1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5)
- **Optimal cost:** 8 moves
- **Manhattan Distance at Start:** 8

Therefore, BFS finds an optimal path with a total movement cost of 8 while avoiding all obstacles.

---

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- **The robot can move up, down, left, right**
- **Some cells are blocked (obstacles) and cannot be traversed**
- **The robot has limited sensing capability (can only observe adjacent cells)**
- **Each movement has a cost of 1, but entering risky zones has an additional cost of 2**
- **The robot must minimize total cost while ensuring safe navigation**
- **The robot must update its knowledge dynamically (online search scenario)**

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

- **Intelligent agents**
- **Rationality**
- **Environment type**

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Autonomous Rescue Robot Using Uniform Cost Search and Online Search

a) Problem Understanding and Constraint Analysis (6 Marks)

An autonomous rescue robot is deployed in a disaster-affected building to locate a survivor. The building is represented as a grid containing free cells, obstacles, and risky zones. The robot has limited sensing capability and must update its knowledge while navigating.

Given Constraints:

1. The robot can move Up, Down, Left, and Right.
2. Obstacles cannot be traversed.
3. The robot can observe only adjacent cells.
4. Each movement costs 1, while entering a risky zone adds an additional cost of 2.
5. The robot must minimize total cost and navigate safely.
6. The robot must update its knowledge dynamically using online search.

Effect of Constraints on Decision-Making:

Constraint	**Effect on Robot**
Four-directional movement	Limits the available actions to four directions.
Obstacles	Prevents movement through blocked cells and forces alternative routes.
Limited sensing	Requires the robot to explore nearby cells before deciding its next move.
Movement cost	Encourages shorter paths.
Risky-zone cost	Encourages safer routes when their total cost is lower.
Dynamic knowledge	Requires the robot to update its internal map after sensing new information.

The robot must balance distance, movement cost, and safety while selecting a path to the survivor.

b) Solution Development Using Uniform Cost Search and Online Search (7.5 Marks)

Chosen Approach: Uniform Cost Search (UCS) combined with an Online Search Agent.

Uniform Cost Search expands the node with the lowest accumulated path cost. It is suitable because movement costs are not equal when risky zones are present. The online agent allows the robot to discover the environment gradually.

Cost Calculation:

- Normal cell movement cost = 1
- Additional risky-zone cost = 2
- Total cost of entering a risky cell = 1 + 2 = 3
- Obstacle movement = Not allowed

The total path cost is:

Total Cost = Normal Movement Costs + Risk Penalties

Steps Involved:

1. Initialize the robot at Start (S), with path cost 0.
2. Sense the adjacent cells to identify free cells, obstacles, and risky zones.

3. Add the newly discovered information to the internal map.
4. Generate valid neighboring cells, excluding known obstacles.
5. Calculate the cumulative cost of reaching each neighbor.
6. Select the reachable unexplored position with the lowest cumulative cost.
7. Move to that position and update the internal map using new sensor information.
8. Repeat the process until the survivor at Goal (G) is reached.
9. If a previously unknown obstacle is discovered, update the map and calculate an alternative route.

Illustrative Grid:

S . R .

. .

. . . G

S = Start, G = Goal, R = Risky zone, # = Obstacle, . = Normal cell.

In this example, the robot must avoid the blocked cells and evaluate whether entering the risky zone is cheaper than taking an alternative route.

Python Program:

```
import heapq

grid = [
    ['S', '.', 'R', '.'],
    ['#', '#', '.', '.'],
    ['.', '.', '.', 'G']
]

start = (0, 0)

goal = (2, 3)

directions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

rows = len(grid)

cols = len(grid[0])

pq = [(0, start)]
```

```
cost = {start: 0}
parent = {start: None}
while pq:
    current_cost, current = heapq.heappop(pq)
    if current_cost != cost[current]:
        continue
    print("Exploring:", current, "Cost:", current_cost)
    if current == goal:
        break
    for dr, dc in directions:
        nr = current[0] + dr
        nc = current[1] + dc
        if not (0 <= nr < rows and 0 <= nc < cols):
            continue
        if grid[nr][nc] == '#':
            continue
        neighbor = (nr, nc)
        move_cost = 3 if grid[nr][nc] == 'R' else 1
        new_cost = current_cost + move_cost
        if new_cost < cost.get(neighbor, float('inf')):
            cost[neighbor] = new_cost
            parent[neighbor] = current
            heapq.heappush(pq, (new_cost, neighbor))
if goal in cost:
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    print("Optimal Path:", path)
```

```
    print("Minimum Cost:", cost[goal])
else:
    print("Goal not reachable")
```

Note: This program assumes the complete grid is known in advance. In a real online-search scenario, the robot must initially mark unobserved cells as unknown and reveal their status through sensing. It should then update its map and replan whenever new information changes the available routes.

c) Application of Artificial Intelligence Concepts (6 Marks)

1. Intelligent Agent

The robot acts as an intelligent agent because it senses its surroundings, processes information, selects actions, and moves toward the survivor.

- Sensors collect information about adjacent cells.
- The internal map stores discovered obstacles and risky zones.
- The search algorithm evaluates available paths.
- Motors execute the selected movement.

2. Rationality

A rational agent selects actions that maximize its expected performance based on available information.

The robot considers:

- Reaching the survivor.
- Minimizing cumulative movement and risk costs.
- Avoiding known obstacles.
- Updating its decisions when new information becomes available.

For example, the robot may choose a longer route through normal cells instead of a shorter route through a risky zone if the longer route has a lower total cost.

3. Environment Type

Environment Property	**Classification**	**Explanation**
Observability	Partially observable	Only adjacent cells can be sensed.
Determinism	Deterministic under the stated model	Actions have predictable outcomes unless additional uncertainty is introduced.
Dynamics	Static or dynamic	The building may be static, but new information becomes available during exploration.
State Space	Discrete	The building consists of individual grid cells.

Agents	Single-agent	One robot makes navigation decisions.
Knowledge	Initially incomplete	The robot discovers obstacles and risky zones while moving.
Costs	Non-uniform	Normal movement costs 1 and risky-zone entry costs 3.

d) Justification of the Chosen Approach (4.5 Marks)

Uniform Cost Search is suitable because the robot must minimize total path cost rather than simply the number of movements. Unlike Breadth-First Search, UCS correctly accounts for additional risky-zone costs.

Combining UCS with online search supports gradual exploration and dynamic map updates. The robot can sense nearby cells, record newly discovered information, avoid obstacles, and replan when its previous route becomes unavailable.

However, UCS alone does not guarantee safety in the real world. Practical deployment requires reliable obstacle detection, risk assessment, and emergency-stop mechanisms. If the environment changes, the robot must update its model and replan accordingly.

Result: Uniform Cost Search combined with an Online Search Agent provides a suitable AI-based approach for navigating a partially observable disaster environment while minimizing cumulative movement and risk costs and adapting to newly discovered information.
